

SESSION 05

깃(Git)이란 무엇인가

Version Control · 자연어로 다루는 버전 관리

WEEK

05

2026.6.08주

점심시간
클로드 캠프

왜 git인가 — 버전 관리가 필요한 순간

02 / 07

ktm_최종.py · ktm_최종_v3.py · ktm_최종_진짜최종.py →

git이 대신한다

필요한 순간 01

되돌리기 ●

잘못 고쳐도 이전 버전으로 한 번에. 망친 걸 손으로 복구할 필요 없음

필요한 순간 02

기록 ●

언제 · 무엇을 · 왜 바꿨는지 모든 변경이 이력으로 남는다

필요한 순간 03

실험 ●

지금 버전을 망칠 걱정 없이 새 시도. 안 되면 그냥 버리면 됨

무엇을 담나

잘 맞음 ✓ 텍스트 파일 — .py .md 설정 (줄 단위로 무엇이 바뀌었는지 비교 가능)

부적합 ✗ 바이너리 — .png .pdf .xlsx 이미지 (비교 안 되고 저장 소만 무거워짐)

git = 폴더의 타임머신

ktm의 scripts · SKILL.md는 git이, 출력 png·보고서는 빼둔다 →

외울 건 딱 3개 — 저장소 · 커밋 · 이력

03 / 07

명령어는 안 외워도 된다

"방금 거 저장해줘"
한 마디면 끝

커밋 이력 (git log)

c3f9a1 새 카테고리 추가
8b2e07 그래프 색 변경
a1b2c3 ktm 첫 버전

동료는 **git commit**을 직접 칠 일이 없다 — 자연어로 시키면
Claude Code가 **git** 명령을 대신 실행

개념 01

저장소 repository

버전이 차곡차곡 쌓이는 폴더. ktm 폴더
를 저장소로 만든다

개념 02

커밋 commit

한 시점의 스냅샷 — 게임의 세이브 포인
트처럼 그 순간을 통째 저장

개념 03

이력 history · log

커밋들이 줄지어 쌓인 기록. 언제든 거슬러 올라가 확인

커밋마다 고유 좌표

a1b2c3d

커밋할 때마다 고유한 커밋 ID(해시)가 남는다. `git log` 로 이력을
확인하고, 이 ID를 기준으로 그 시점으로 되돌리기(revert) — "그때
로 돌아가줘"의 좌표

ktm 폴더를 저장소로 만들기

04 / 07

CLAUDE CODE에 그대로 입력

이 폴더 **[ktm 폴더 경로]**를
git으로 버전 관리 시작해줘.

→ Claude가 뒤에서 한 번에 처리:

```
git init → git add . → git commit -m "ktm 첫 버전"
```

= 저장소 생성 + 첫 커밋까지 완료

함께 만들면 좋은 것

.gitignore — 추적 안 할 목록

```
# 출력물·바이너리
outputs/
*.png
*.pdf
# 캐시·임시
__pycache__/
.DS_Store
# 비밀번호
.env
```

"ktm에 .gitignore 만들어줘" 한 줄이면 채워줌

사전 준비

① git 설치 — 자리에서 "git 설치해줘" 한 줄 (3주차 python과 같은 방식) · ② 대상 = week4에서 만든 ktm 폴더

※ 직접 입력 — 저장소가 만들어지는 과정을 함께 본다

약 3분

실습 — ktm을 더 개발하며 버전 남기기

05 / 07

개선 발화 (예시)

"ktm 그래프 색 바꾸고
새 카테고리 추가해줘"

고치고 → 저장 → 확인 → 되돌리기

· 한 바퀴 돌며 버전이 쌓이는 걸 직접 본다

CLAUDE CODE 커밋 팁

"주요 작업 할 때마다
안 시켜도 알아서 커밋해"

한 번 일러두면 매번 "저장해줘" 안 해도 단계마다 자동으로 버전이 쌓인다

1

개선

ktm 한 군데 수정 — 그래프 색.새
category

scripts 수정

2

저장 (커밋)

"방금 바꾼 거 저장해줘"

git commit

3

이력 확인

"지금까지 뭐 바꿨는지 보여줘"

git log

4

되돌리기

"아까 버전으로 되돌려줘"

git restore / revert

한 줄 발화 → 버전으로 안전하게 누적

망쳐도 되돌릴 수 있으니 마음껏 고친다

이 발화 몇 개면 충분 — 더 과감하면 브랜치

06 / 07

동료의 말 → 뒤에서 도는 GIT

"방금 거 저장해줘" → `git add + commit`

"뭐가 바뀌었어?" → `git status / diff`

"지금까지 이력 보여줘" → `git log`

"아까로 되돌려줘" → `git restore / revert`

"실험용으로 따로 해보자" → `git branch / switch`

결다리 · 실험 안전망

브랜치 = 실험용 평행 우주

 본 버전 \ 실험 버전

본 버전은 그대로 둔 채 따로 시도. 잘 되면 합치고, 아니면 그냥 버림

"ktm 실험 버전 만들어줘"

외울 명령은 없다 — 평소 말로 시키면 된다

다 로컬에서 일어나는 일 · 인터넷 필요 없음

TODAY

버전 관리 감각을 익힘 ✓

저장소 · 커밋 · 이력 · 되돌리기

외운 건 명령어가 아니라 **되돌릴 수 있다**는 **안심** — 그러니 마음껏 고친다. 자연어 한 줄이면 Claude가 git을 대신 다룬다.

동료 본인의 폴더도 똑같이 — 보고서·자료 폴더도 "git으로 관리해줘" 한 줄이면 같은 타임머신이 생긴다

다음 주차 · 6월15일주

에이전트와 서브 에이전트

한 명의 Claude가 아니라 여러 일꾼을 나눠 동시에 시키기.

이어서 받는 누적 흐름



git 개념이 아직 헷갈려도 걱정할 것 없다 — 6월22일주에 깃허브(GitHub)를 배우고 나면 한결 쉽게 느껴진다. 차차 **learning by doing**, 직접 해 보며 익히면 된다.